



Semester 1 Examinations 2014 / 2015

Exam Code(s)	1SD1/1SD3/1MF1/1IT1
Exam(s)	Higher Diploma in Applied Science (Software Design & Development), Higher Diploma in Applied Science (Software Design & Development) Industry Stream, M.Sc. in Software Design & Development, Master of Information Technology
Module Code(s)	CT874
Module(s)	Programming I
Paper No.	I
External Examiner(s)	Professor Liam Maguire
Internal Examiner(s)	Professor Gerard Lyons Dr. Michael Madden * Seamus Hill
<u>Instructions:</u>	Candidates are required to answer: Section A (20 Marks) Answer all 10 parts of section A. This section consists of 10 multiple-choice questions, all of which should be attempted. Answers are to be written on the MCQ sheet provided, NOT on the Examination paper. Section B (80 Marks) Answer Any Two Questions.
Duration	2 hours
No. of Pages	7
Department(s)	Information Technology
Requirements	MCQ

Section A (20 Marks)
Attempt all 10 questions
Mark your answers on the MCQ sheet provided

Q1.

1) How would we write the first line of a class, Toyota, such that it extends the class, Car?

- a) public Car extends Toyota{
- b) public Toyota extends Car{
- c) public Toyota implements Car{
- d) public Car implements Toyota {

2) Consider the following code:

```
class X{
    public void parentMethod(){
        System.out.println("parent");
    }
}

class Y extends X{
    public static void main(String args[]){
        X x = new X();
        x.parentMethod();
    }

    public static void myMethod(){
        System.out.println("child");
    }
}
```

What is the output of this code?

- a) parent
- b) child
- c) X
- d) There is no output

3) Which visibility modifier allows the data members of a superclass to be accessible to the instances of subclasses only?

- a) public
- b) private
- c) packaged
- d) protected

PTO

4) In the following program:

```
class Truck extends Vehicle{  
  
    public Truck(){  
  
        super();  
  
    }  
  
}
```

What does super do?

- a) Makes a call to the Object() constructor.
- b) Makes a call to the Truck() constructor.
- c) Makes a call to the Vehicle() constructor.
- d) Makes a call to the superclass of Vehicle's constructor.

5) What happens when the following code is run?

```
String a = "illuminati";  
  
for (int i = 1; i < 10; i++) {  
    System.out.print(a.charAt(i));  
}
```

- a) The output will be "illuminati"
- b) The output will be illuminati
- c) The output will be lluminati
- d) An exception will be thrown

6) Assume we have an array of doubles called someArray, what does the following program do?

```
double x = 0, unknown;  
  
for (int j = 0; j < someArray.length; j++){  
    x+=someArray[j];  
}  
unknown = x /someArray.length;
```

- a) Computes the sum of the doubles in the array, someArray.
- b) Deletes all the doubles in the array, someArray.
- c) Computes the average of all the doubles in the array, someArray.
- d) Computes the dot product of all the doubles in the array, someArray.

PTO

7) How would we access the 12th row of the 3rd column of the two-dimensional array, x?

- a) x[12][3];
- b) x[11][2];
- c) x[3][12];
- d) x[2][11];

8) A(n) _____ has a method prototype but no body.

- a) default constructor
- b) default method
- c) concrete method
- d) abstract method

9) What will be the output from the following code?

```
try {  
    int num1 = Integer.parseInt("130");  
    Sytstem.out.println("First is okay");  
    int num2 = Integer.parseInt("i95");  
    System.out.println("Second is okay");  
} catch (NumberFormatException e) {  
    System.out.println("Error");  
}
```

- a) First is okay Second is okay
- b) First is okay Second is okay Error
- c) Error
- d) First is okay Error

10) What is wrong with the following block of code? (assume Bag is defined).

```
class New {  
    Bag b;  
    public void getBag(){  
        return b;  
    }  
}
```

- a) The accessor method needs a different return value.
- b) b has not yet been initialized.
- c) The class needs more methods.
- d) The method getBag needs more statements.

PTO

Section B (80 Marks)
Answer Any TWO Questions

Q 2.

- a) In relation to object-oriented programming with Java, detail your understanding, using examples/analogies where appropriate, of each of the following:

- i. Polymorphism
- ii. Interface
- iii. Abstract class
- iv. Inheritance

[12]

- b) Write a Java application, which uses a two-dimensional array to create and store a guest list of first and last names of people attending a party. The array should store all of the first names in one column and all of the second names in another column. The application should also display the contents of the array.

[15]

- c) Create a Java application that simulates the rolling of a pair of dice ten times and stores the sum of the die for each roll in an array.

[13]

Q 3.

- a) Explain the terms **overloading** and **overriding** as used in object-oriented programming. You should provide snippets of Java code to illustrate your answer.

[10]

- b) Using the following interface, you are required to develop a Java application for the *Rare Teddy Bear Company*, which will allow you to store information relating to the companies stock of teddy bears. The company stores the name of the manufacturer, the colour of the bear and the year of manufacture.

```
public interface TeddyBearInterface {  
  
    public String getManufacturer();  
    public void setManufacturer(String maker);  
    public String getColour();  
    public void setColour(String col);  
    public int getYearOfManufacture();  
    public void setYearOfManufacture(int year);  
}
```

You are required to:

- i. Create a ***TeddyBear*** class using the above interface as a specification for the desired methods. The *TeddyBear* class should overload the constructors and include an overridden implementation of the *toString* method.

[10]

- ii. Write a **test class** for the *TeddyBear* class called ***TeddyBearTest***. The test class should store each individual instance of *TeddyBear* created into an **Array List**. The test class class should include the following:
- a method for **adding** *TeddyBear* objects to the array list
 - a method for **removing** *TeddyBear* objects from the array list
 - a method for **displaying** the contents of the array list.

[20]

Q 4.

Pet Planet seek an application to keep track of their pets and want you to create an initial Java program to illustrate the inheritance hierarchy. Give the following abstract class,

```
abstract class Animal {
    private String name;

    public Animal(){
        name = null;
    }
    public void setName(String animalName){
        name = animalName;
    }
    public String getName(){
        return name;
    }
    abstract String sound();
}
```

you are required to:

- a) Create a **Dog** class which inherits from the **Animal** class and includes a **fetch** method which prints out the string "I am fetching a " with the item passed as a parameter, concatenated to it. The method has the following header:

```
public void fetch(String item)
```

[8]
- b) Create a **Cat** class which inherits from the **Animal** class and includes a **climb** method which prints out the string "I'm climbing a " with the item passed as a parameter, concatenated to it. The method has the following header:

```
public void climb(String item)
```

[8]
- c) Create a class called **Beagle**, which is a sub class to the **Dog** class and implements the **sound** method, which returns the string "woof woof".

[8]
- d) Create a class called **Saimese**, which is a sub class of the **Cat** class and implements the **sound** method, which returns the string "meow meow".

[8]
- e) You are also required write a **test class** called **AnimalTest** which:
 - creates an instance of the Beagle class and sets the **name** of the dog, calls the objects **sound** method and it's **fetch** method.

[4]
 - creates an instance of the Siamese class and sets the **name** of the cat, calls the objects **sound** method and it's **climb** method.

[4]